Angular Memory Leak Lab

Practical Report: how a memory leak happens in Angular, and how to fix it

1. Objective

This practical shows how a memory leak happens in an Angular application and how to fix it. The leak is caused by a browser **window** event listener that is not removed when an Angular component is destroyed. I built two versions of one component, a leaking one and a fixed one, and compared them using the app's own logs, Chrome DevTools, and an inspector page.

2. Basic Concepts

Term	Simple meaning
Angular component	A small part of an application, such as a card or a panel.
Mounting / unmounting	Mounting means creating and showing a component. Unmounting means removing it from the screen.
Event listener	Code that runs when an event happens, for example when the browser window is resized.
Memory leak	Objects stay in memory because something still points to them, even though the app no longer needs them.
Garbage Collector	JavaScript's automatic cleanup. It frees objects that nothing points to anymore.
Heap Snapshot	A picture of the JavaScript objects in memory at one moment.
Retainers	The references that explain why an object is still kept in memory.
ngOnDestroy()	The Angular method that runs when a component is destroyed. It is the place to do cleanup.

3. Practical Implementation

A. Leaking Component

- It adds a resize listener with `window.addEventListener()`.
- It does **not** remove the listener in `ngOnDestroy()`.
- After the component is removed, the listener still runs.
- The listener points to the component, so the component and its payload data can stay in memory.

```
window.addEventListener('resize', ...)
```

B. Fixed Component

- It saves the listener and removes it with `window.removeEventListener()` inside `ngOnDestroy()`.
- This breaks the unwanted reference.
- The component and its data can now be garbage-collected, if nothing else points to them.

```
window.removeEventListener('resize',
this.handler)
```

The two code lines are the ones shown in the app's dashboard descriptions.

4. Testing and Screenshot Evidence

Step 1 - Dashboard. The dashboard has the Leaking Component and the Fixed Component side by side. Each has an Open button and a Cycle 5x (Open and Close) button. In the screenshot, both are unmounted (destroyed).

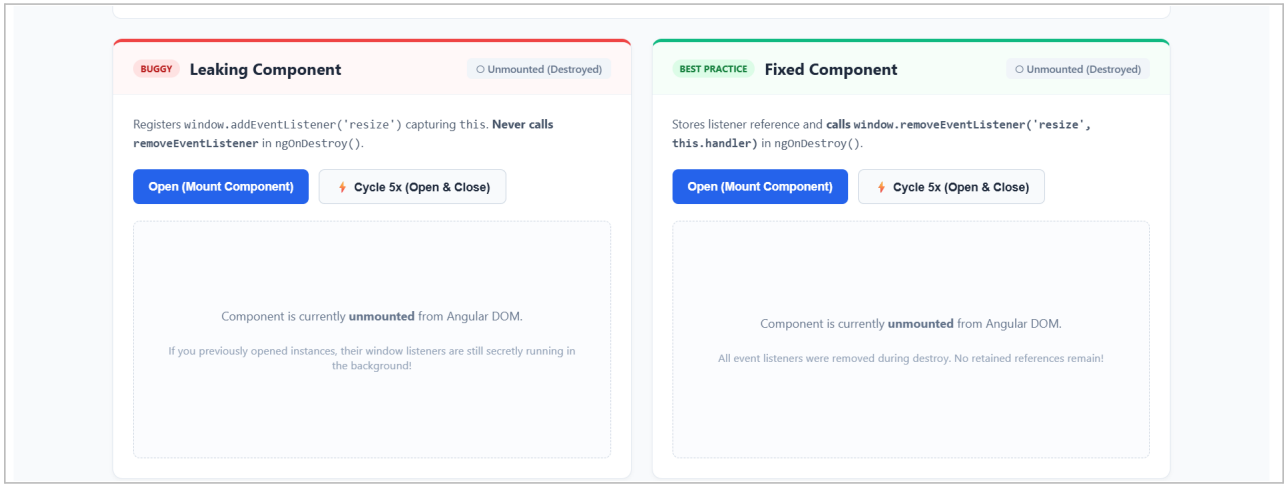


Figure 1: Leaking and Fixed components side by side. Both are now removed from the screen. The text on each card explains the difference.

Step 2 - Zombie callback logs. After the components were destroyed, a window resize happened. The logs show a warning for each leaking component (#1 to #5): the component is unmounted, but its resize listener is still running.

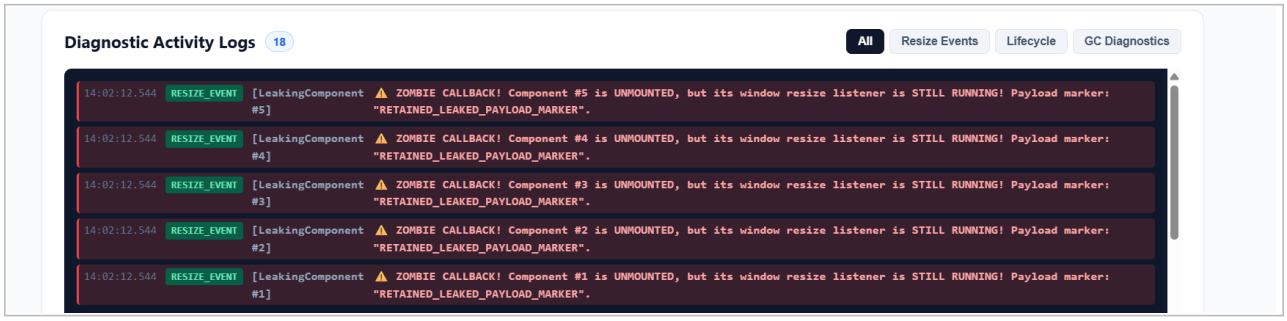


Figure 2: Zombie callbacks. Destroyed leaking components #1 to #5 still react to the resize event.

Step 3 - Heap Snapshot. In Chrome DevTools (Memory tab) I took a heap snapshot and searched for the payload class. It lists five `LeakedPayload` objects, one for each leaking component I opened.

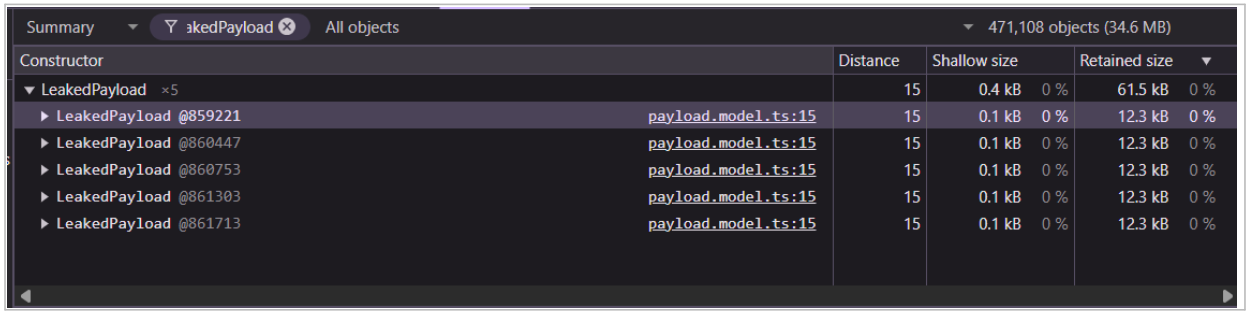


Figure 3: Heap Snapshot with five LeakedPayload objects still in memory.

Step 4 - Retainers. For the selected LeakedPayload, the Retainers panel shows what is holding it. At the bottom of the list is `windowResizeHandler` in `LeakingDemoComponent` (`leaking-demo.component.ts:16`). This points to the resize handler as a reason the objects are still kept.

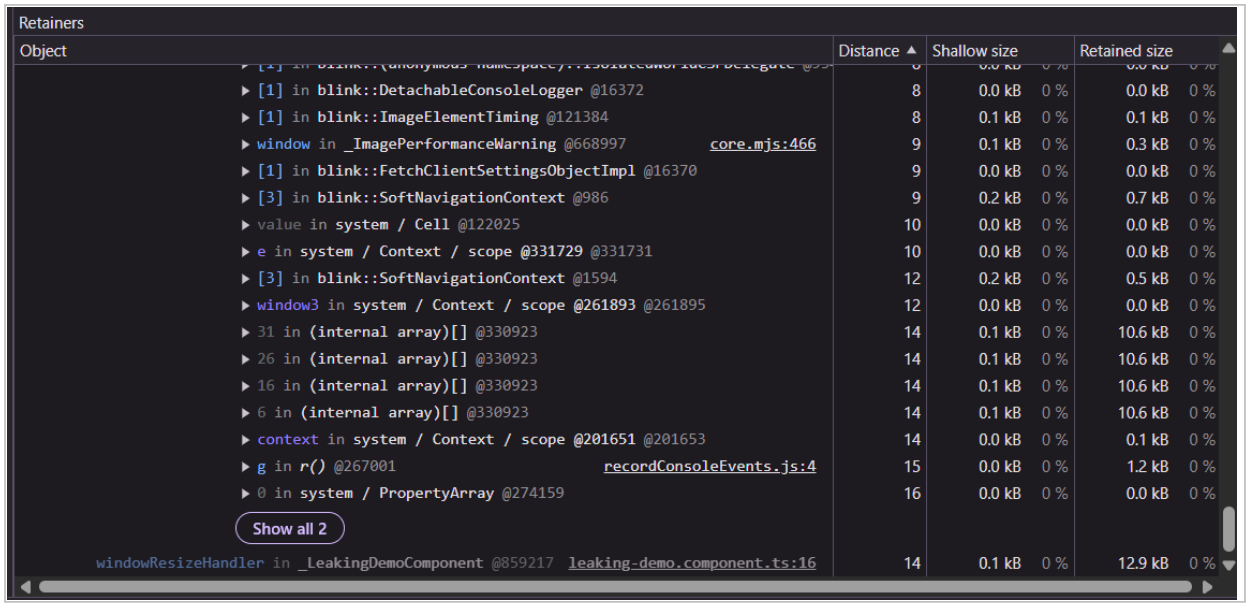


Figure 4: Retainers evidence. windowResizeHandler in LeakingDemoComponent appears in the retaining list.

Step 5 - Final comparison. The Object Lifetime Inspector uses `WeakRef` (a reference that does not keep an object alive) and `FinalizationRegistry` (tells us when an object was cleaned up). It shows the fixed and leaking instances together.

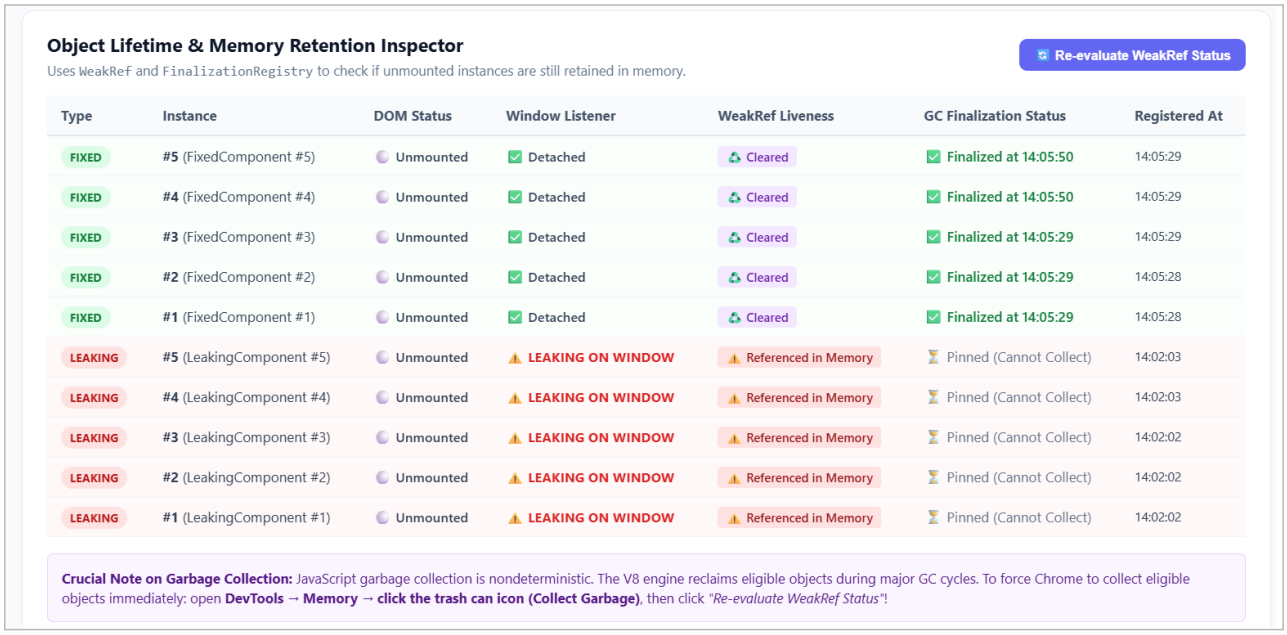


Figure 5: Fixed instances (#1 to #5) are Detached, Cleared and Finalized. Leaking instances (#1 to #5) are LEAKING ON WINDOW and Referenced in Memory.

5. Results and Conclusion

- The leaking component's resize listener kept running after the component was removed (Figure 2).
- The heap snapshot found five `LeakedPayload` objects, and the Retainers panel points to `windowResizeHandler` (Figures 3 and 4).
- The fixed component removes its listener in `ngOnDestroy()`.
- The inspector reports the fixed instances as cleared and finalized, while the leaking instances are still referenced (Figure 5).

Note: JavaScript decides when garbage collection happens, so it is not guaranteed to be immediate. This practical shows retained objects and callbacks in one controlled example. It does not prove long-term memory growth.